

UNIVERSITATEA TEHNICĂ A MOLDOVEI
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică

LUCRARE DE LABORATOR NR. 6
Inside the xv6 File System

Disciplina: Sisteme de Operare

A efectuat:
student gr. FAF-242
Daniel-Ioan Strunga

Au verificat:
asist. univ. Patricia Reițman
asist. univ. Marius Dumitrașcu

Chișinău — 2026

Cuprins

1 Part 1. Read what xv6 shows you **2**

2 Part 2. Write `mystat` **2**
2.1 Codul sursă (`user/mystat.c`) 2
2.2 Rulare în xv6 și inspecția codului din kernel 3

3 Part 3. Measure the limits **4**

4 Part 4. Write `find` **5**
4.1 Codul sursă (`user/find.c`) 5
4.2 Rulare de test în xv6 6

5 Concluzie **7**

1 Part 1. Read what xv6 shows you

```
user@user-VPCEH1L0E:~/xv6-riscv$ cat kernel/stat.h
#define T_DIR     1   // Directory
#define T_FILE    2   // File
#define T_DEVICE  3   // Device

struct stat {
  int dev;     // File system's disk device
  uint ino;    // Inode number
  short type;  // Type of file
  short nlink; // Number of links to file
  uint64 size; // Size of file in bytes
};
```

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35376
echo           2 4 34264
...
console        3 25 0
$ mkdir a
$ ls a
.              1 28 32
..             1 1 1024
```

Răspunsuri (Observe Part 1):

1. Conform fișierului `kernel/stat.h`, constantele de tip reprezintă:
   - `1` (`T_DIR`) — Director (*Directory*);
   - `2` (`T_FILE`) — Fișier obișnuit (*Regular file*);
   - `3` (`T_DEVICE`) — Dispozitiv special I/O (*Device*).
2. Directorul rădăcină (`/`) are numărul de inode `1` (`ROOTINO`). În rădăcină, intrările `.` și `..` au ambele inode-ul `1` deoarece directorul rădăcină nu are un alt director părinte deasupra sa în ierarhia arborelui, deci este propriul său părinte.
3. Un director nou creat (`a`) are dimensiunea de **32 bytes** și conține exact două intrări implicite: `.` (referința către sine) și `..` (referința către părinte, inode 1). Prin urmare, o singură intrare de director (`struct dirent`) are dimensiunea fixă de 32/2 = **16bytes** (2 bytes pentru `ushort inum` + 14 bytes pentru `char name[DIRSIZ]`).

2 Part 2. Write `mystat`

2.1 Codul sursă (`user/mystat.c`)

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
```

```
4
5  char *
6  typename(int t)
7  {
8    if(t == T_DIR) return "directory";
9    if(t == T_FILE) return "file";
10   if(t == T_DEVICE) return "device";
11   return "?";
12 }
13
14 int
15 main(int argc, char *argv[])
16 {
17   struct stat st;
18   for(int i = 1; i < argc; i++){
19     if(stat(argv[i], &st) < 0){
20       fprintf(2, "mystat: cannot stat %s\n", argv[i]);
21       continue;
22     }
23     printf("%s: %s, inode %d, links %d, size %d bytes\n",
24            argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
25   }
26   exit(0);
27 }
```

2.2 Rulare în xv6 și inspecția codului din kernel

```
1  $ mystat README . console
2  README: file, inode 2, links 1, size 2441 bytes
3  .: directory, inode 1, links 1, size 1024 bytes
4  console: device, inode 25, links 1, size 0 bytes
5  $ ln README r2
6  $ mystat README r2
7  README: file, inode 2, links 2, size 2441 bytes
8  r2: file, inode 2, links 2, size 2441 bytes
9  $ rm README
10 $ mystat r2
11 r2: file, inode 2, links 1, size 2441 bytes
12 $ cat r2 | wc
13 46 318 2441
```

```
1  user@user-VPCEH1L0E:~/xv6-riscv$ grep -n -C 5 "ip->nlink--" kernel/
   sysfile.c
2  239-    dp->nlink--;
3  240-    iupdate(dp);
4  241-  }
5  242-  iunlockput(dp);
6  243-
7  244:  ip->nlink--;
8  245-  iupdate(ip);
9  246-  iunlockput(ip);
10
11 user@user-VPCEH1L0E:~/xv6-riscv$ grep -n -A 8 "struct dinode {" kernel/
   fs.h
12 32:struct dinode {
13 33-  short type;               // File type
```

```
34-  short major;              // Major device number (T_DEVICE only)
35-  short minor;              // Minor device number (T_DEVICE only)
36-  short nlink;              // Number of links to inode in file system
37-  uint size;                // Size of file (bytes)
38-  uint addrs[NDIRECT + 1]; // Data block addresses
39-};
```

Răspunsuri (Observe Part 2):

1. După comanda `ln README r2`, ambele nume afișează exact același număr de inode (2), aceeași dimensiune (`2441 bytes`) și același tip (`file`), însă contorul `links` a crescut de la 1 la 2. După `rm README`, fișierul `r2` își păstrează numărul de inode (2) și conținutul intact (`2441 bytes`), iar contorul `links` revine la 1.

2. În `kernel/sysfile.c` (în funcția `sys_unlink`), linia **244** (`ip->nlink-;`) decrementează numărul de legături ale inode-ului țintă. Blocurile de date și inode-ul pe disc sunt eliberate efectiv prin funcția `itrunc(ip)` apelată din `iput()` (în `kernel/fs.c`) doar în momentul în care atât `ip->nlink == 0` (nu mai există niciun nume pe disc), cât și `ip->ref == 0` (niciun proces nu mai ține fișierul deschis în memorie).

3. Conform structurii `struct dinode` din `kernel/fs.h` (liniile 32–39), un inode pe disc în xv6 conține exclusiv 6 câmpuri:

   - `short type` — tipul inode-ului (liber, director, fișier, dispozitiv);
   - `short major` și `short minor` — identificatorii pentru dispozitivele hardware (`T_DEVICE`);
   - `short nlink` — numărul de legături fizice (hard links) către acest inode;
   - `uint size` — dimensiunea fișierului în bytes;
   - `uint addrs[NDIRECT + 1]` — vectorul cu adresele blocurilor de date (12 blocuri directe + 1 bloc simplu indirect).

3 Part 3. Measure the limits

```
user@user-VPCEH1L0E:~/xv6-riscv$ grep -n "define BSIZE\|define NDIRECT\|
    define NINDIRECT\|define DIRSIZ" kernel/fs.h
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14

user@user-VPCEH1L0E:~/xv6-riscv$ make fs.img 2>&1 | grep nmeta
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1)
    blocks 1953 total 2000
```

```
$ mkdir averyveryverylongname
$ ls
...
averyveryveryl 1 29 32
```

Răspunsuri (Observe Part 3):

1. Comanda `ls` afișează numele trunchiat la primele 14 caractere: `averyveryveryl`. Această limitare este explicată de constanta `#define DIRSIZ 14` (linia 54 din `kernel/fs.h`), care fixează lungimea maximă a numelui într-o intrare de director (`struct dirent`) la 14 octeți.

2. **Calculul dimensiunii maxime a unui fișier în xv6:**

   - Dimensiunea unui bloc: `BSIZE` $= 1024 bytes = 1KB$;
   - Numărul de blocuri directe: `NDIRECT` $= 12$;
   - Numărul de adrese stocate în blocul simplu indirect: `NINDIRECT` $= 1024/4 = 256 blocuri$;
   - Total blocuri de date per fișier: $12 + 256 = 268 blocuri$;
   - Dimensiunea maximă în bytes: $268 \times 1024 bytes =$ **274 432bytes** (exact **268 KB**).

3. Conform liniei `nmeta` afișate de utilitarul `mkfs` la construirea imaginii `fs.img`:

   - **Total blocuri pe disc:** `2000` blocuri (Dintre care `47` sunt blocuri de metadate: 1 bloc de boot, 1 superblock, **31** blocuri pentru jurnal/log, **13** blocuri pentru inodes și 1 bloc pentru bitmap);
   - **Blocuri de date (data blocks):** `1953` blocuri $(2000 - 47 = 1953)$.

4 Part 4. Write `find`

4.1 Codul sursă (`user/find.c`)

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"

void
find(char *path, char *target)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(path, 0)) < 0){
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }

  if(fstat(fd, &st) < 0){
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd);
    return;
  }

  if(st.type == T_DIR){
    if(strlen(path) + 1 + DIRSIZ + 1 > sizeof buf){
```

```
27        printf("find: path too long\n");
28        close(fd);
29        return;
30      }
31      strcpy(buf, path);
32      p = buf + strlen(buf);
33      *p++ = '/';
34      while(read(fd, &de, sizeof(de)) == sizeof(de)){
35        if(de.inum == 0)
36          continue;
37        if(strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
38          continue;
39        memmove(p, de.name, DIRSIZ);
40        p[DIRSIZ] = 0;
41        if(stat(buf, &st) < 0){
42          printf("find: cannot stat %s\n", buf);
43          continue;
44        }
45        if(strcmp(de.name, target) == 0){
46          printf("%s\n", buf);
47        }
48        if(st.type == T_DIR){
49          find(buf, target);
50        }
51      }
52    }
53    close(fd);
54  }
55
56  int
57  main(int argc, char *argv[])
58  {
59    if(argc < 3){
60      fprintf(2, "usage: find <dir> <name>\n");
61      exit(1);
62    }
63    find(argv[1], argv[2]);
64    exit(0);
65  }
```

4.2 Rulare de test în xv6

```
1  $ mkdir a
2  $ mkdir a/b
3  $ echo x > a/b/r2
4  $ echo y > a/r2
5  $ ln cat r2
6  $ find . r2
7  ./a/b/r2
8  ./a/r2
9  ./r2
10 $ find a b
11 a/b
```

5 Concluzie

În cadrul acestei lucrări de laborator am analizat în profunzime implementarea internă a sistemului de fișiere din nucleul xv6 (`kernel/fs.h`, `kernel/stat.h` și `kernel/sysfile.c`). Am dezvoltat utilitarul de spațiu utilizator `mystat.c` pentru a interoga direct metadatele inode-urilor prin apelul de sistem `stat()` și am urmărit mecanismul de funcționare a legăturilor fizice (*hard links*) și a eliberării blocurilor la `unlink`. De asemenea, am determinat limitele structurale ale sistemului de fișiere xv6 (lungimea numelui `DIRSIZ = 14`, dimensiunea maximă a unui fișier de 268 KB și structura celor 2000 de blocuri ale imaginii discului) și am implementat programul recursiv `find.c`, care parcurge arborele de directoare citind intrările brute `struct dirent` și evitând buclele infinite generate de referințele `.` și `..`